

Sentence Memorability Graphs

```
raw_dataset <- read_csv(
  "sentence_memorability_data.csv",
  na = c("", "NA", "N/A"),
  show_col_types = FALSE
)

# 1. Assign Block Numbers using 5-minute gaps
time_blocked_data <- raw_dataset %>%
  # 1. Sort the data chronologically per participant
  arrange(participant_ID, Timestamp) %>%
  group_by(participant_ID) %>%
  mutate(
    # Make sure Timestamp is numeric (sometimes CSVs read them as text)
    Timestamp_num = as.numeric(Timestamp),

    # 2. Calculate the gap between the current row and the previous row
    # (default = first(Timestamp_num) prevents NA on the very first row)
    time_diff_ms = Timestamp_num - lag(Timestamp_num, default = first(Timestamp_num)),

    # 3. Check if the gap is greater than 300,000 ms (5 minutes)
    is_new_block = time_diff_ms > 60000,

    # 4. cumsum() counts how many times the 5-min gap happened.
    # Adding 1 ensures the first chunk starts at Block 1 instead of 0.
    Block_Number = cumsum(is_new_block) + 1,

    # Your existing formatting logic
    Accuracy_IR = suppressWarnings(as.numeric(Accuracy_IR)),
    Accuracy_WR = suppressWarnings(as.numeric(Accuracy_WR)),
    prev_IR = lag(Accuracy_IR)
  ) %>%
  ungroup() %>%
  # Filter to just your target trials AFTER assigning the blocks
  filter(isTarget == TRUE)

# 2. Aggregate scores per Participant AND Block
blocked_aggregated <- time_blocked_data %>%
  group_by(participant_ID, Block_Number) %>%
  summarise(
    # IR components
    Total_Repeats = sum(isRepeat == TRUE & Event == "Sentence shown", na.rm = TRUE),
    Total_Denom = sum(is.na(isRepeat) & Event == "Sentence shown", na.rm = TRUE),
    Hits = sum(Event == "IR pressed" & isRepeat == TRUE & Accuracy_IR == 1, na.rm = TRUE),
    False_Alarms = sum(Event == "IR pressed" & is.na(isRepeat), na.rm = TRUE),

    # IR metrics
    Hit_Rate = if_else(Total_Repeats > 0, Hits / Total_Repeats, 0),
```

```

False_Alarm_Rate = if_else(Total_Denom > 0, False_Alarms / Total_Denom, 0),
IR_Memorability = Hit_Rate - False_Alarm_Rate,

# WR components
WR_correct = sum(Event == "WR pressed" & Accuracy.WR == 1 & prev_IR == 1, na.rm = TRUE),
WR_total   = sum(Event == "WR pressed" & prev_IR == 1, na.rm = TRUE),

# WR metric
WR_Accuracy = if_else(WR_total > 0, WR_correct / WR_total, 0),
.groups = "drop"
) %>%
# Ensure the Block_Number is a categorical factor for plotting/testing
# (If some participants had more than 3 blocks due to pausing, you can filter them here)
filter(Block_Number %in% c(1, 2, 3)) %>%
mutate(Block_Number = factor(Block_Number, levels = c(1, 2, 3), labels = c("Block 1", "Block 2", "Block 3")))

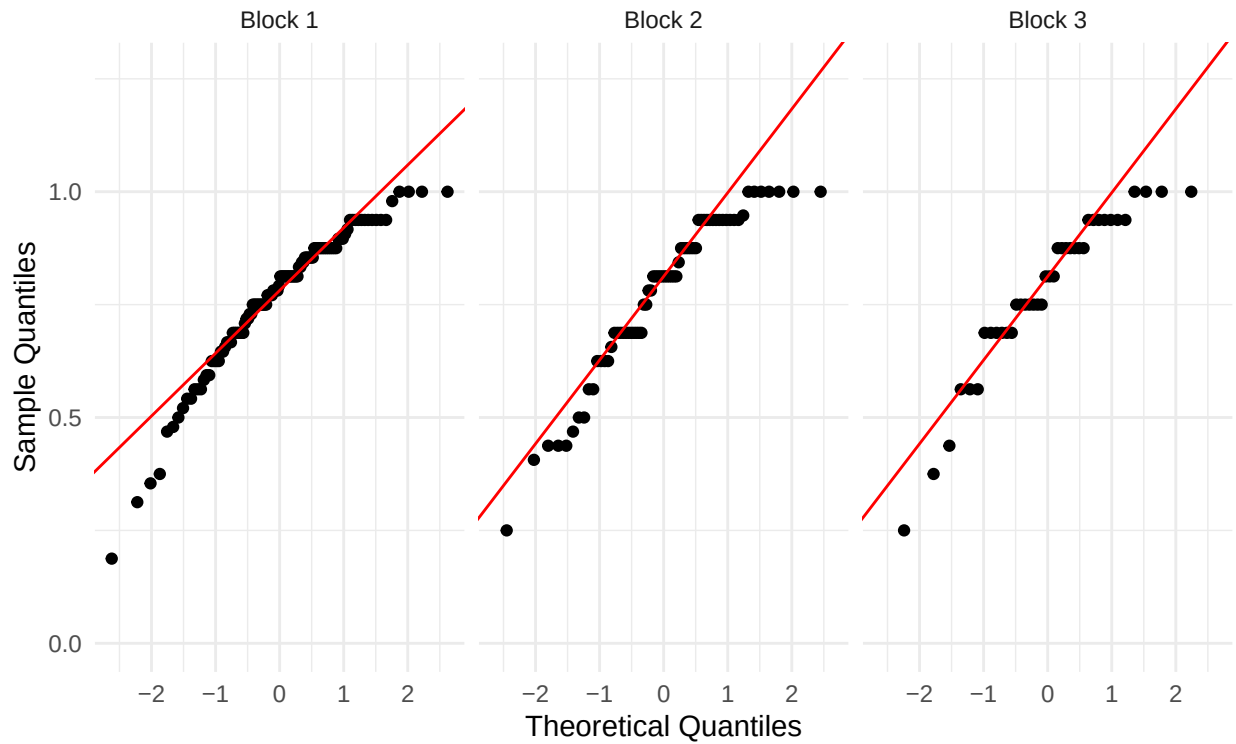
library(ggplot2)

# Q-Q Plot for IR Memorability by Block
ggplot(blocked_aggregated, aes(sample = IR_Memorability)) +
  stat_qq() +
  stat_qq_line(color = "red") +
  facet_wrap(~ Block_Number, ncol = 3) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(title = "Q-Q Plots: IR Memorability Across Blocks",
       subtitle = "Checking for Normality in Blocks 1, 2, and 3",
       x = "Theoretical Quantiles",
       y = "Sample Quantiles") +
  theme_minimal()

```

Q-Q Plots: IR Memorability Across Blocks

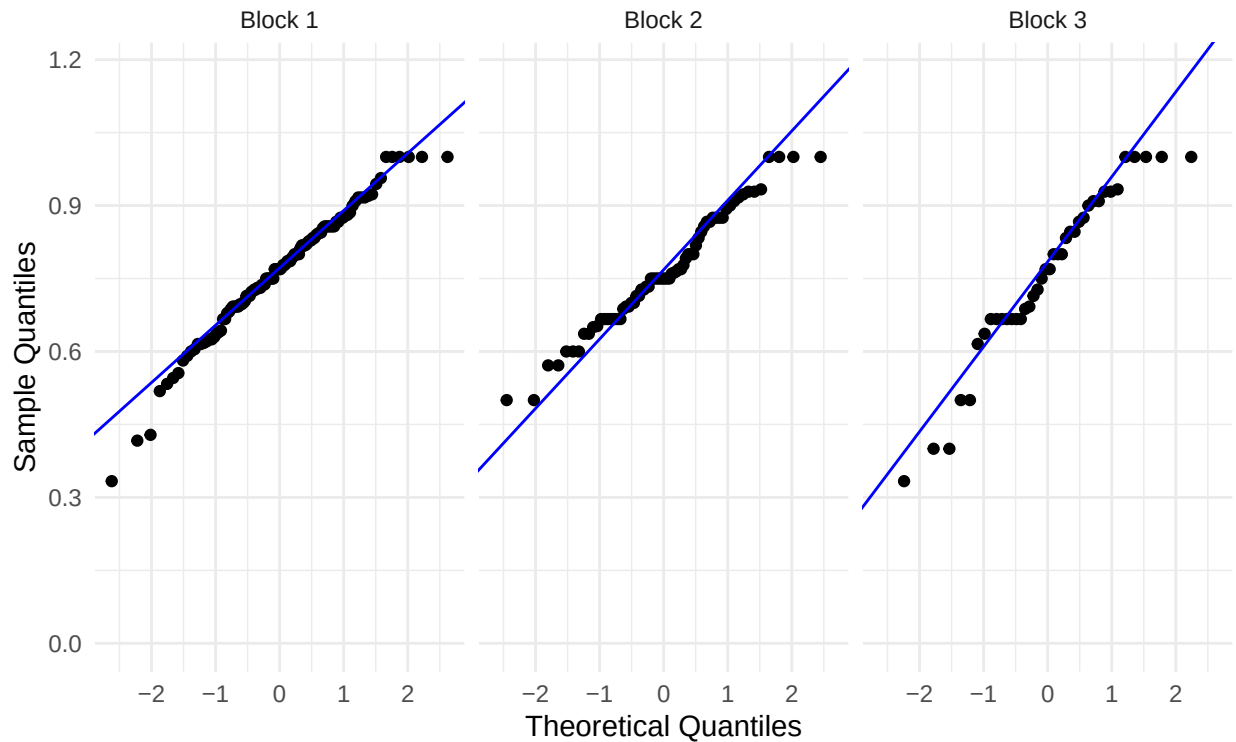
Checking for Normality in Blocks 1, 2, and 3



```
# Q-Q Plot for WR Accuracy by Block
ggplot(blocked_aggregated, aes(sample = WR_Accuracy)) +
  stat_qq() +
  stat_qq_line(color = "blue") +
  facet_wrap(~ Block_Number, ncol = 3) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(title = "Q-Q Plots: WR Accuracy Across Blocks",
       subtitle = "Checking for Normality in Blocks 1, 2, and 3",
       x = "Theoretical Quantiles",
       y = "Sample Quantiles") +
  theme_minimal()
```

Q-Q Plots: WR Accuracy Across Blocks

Checking for Normality in Blocks 1, 2, and 3

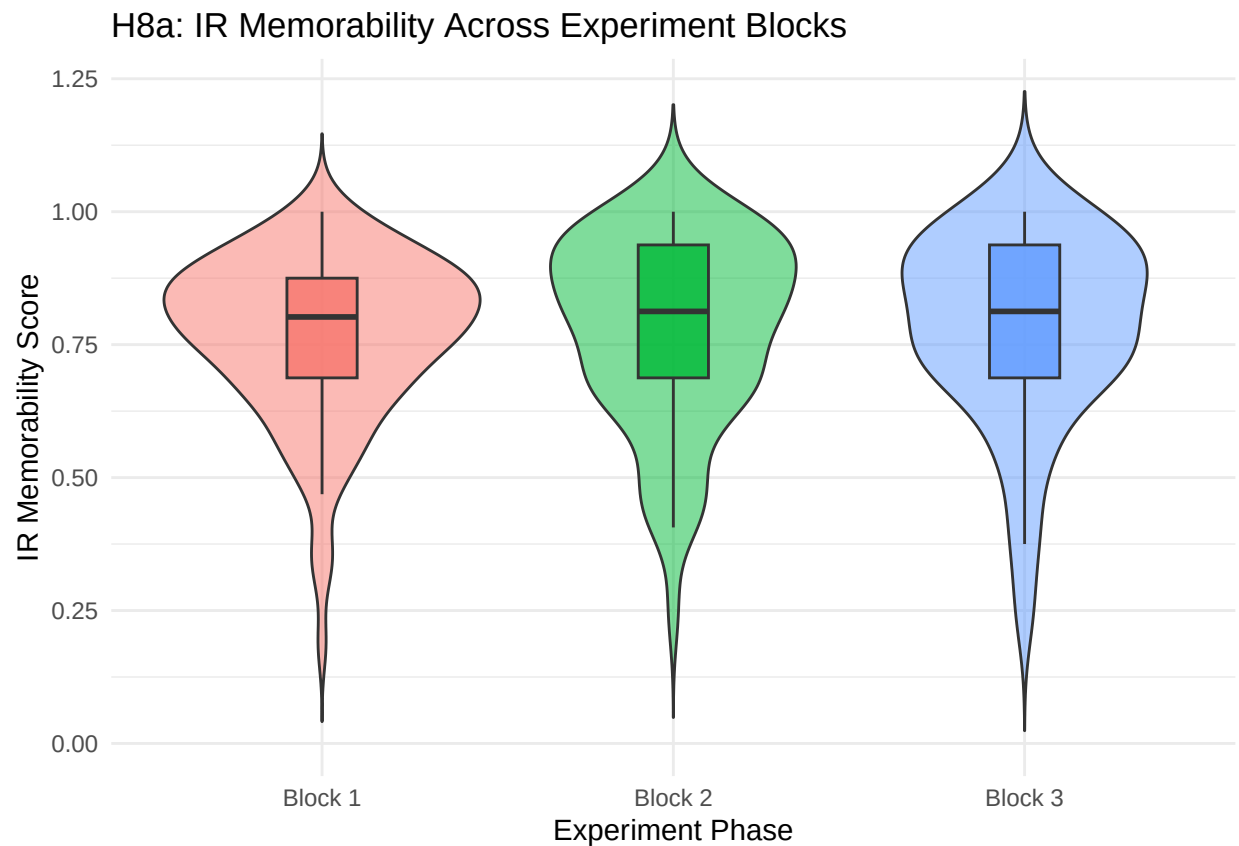


```
# Anderson Darling
library(nortest)

ir_normality_blocks <- blocked_aggregated %>%
  group_by(Block_Number) %>%
  summarise(
    AD_Stat = ad.test(IR_Memorability)$statistic,
    p_value = ad.test(IR_Memorability)$p.value
  )

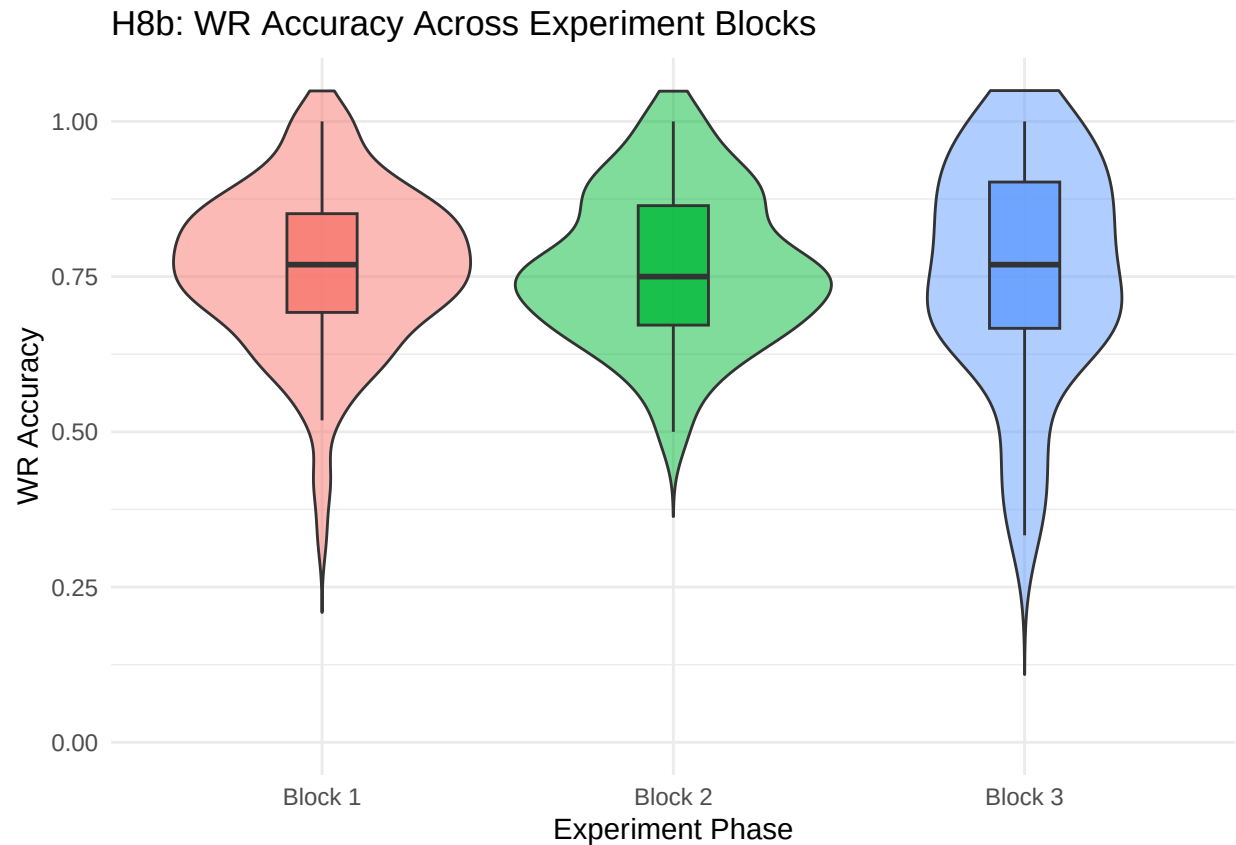
wr_normality_blocks <- blocked_aggregated %>%
  group_by(Block_Number) %>%
  summarise(
    AD_Stat = ad.test(WR_Accuracy)$statistic,
    p_value = ad.test(WR_Accuracy)$p.value
  )

# Graph H8a: Violin & Boxplot for IR Memorability across Blocks
ggplot(blocked_aggregated, aes(x = Block_Number, y = IR_Memorability, fill = Block_Number)) +
  geom_violin(trim = FALSE, alpha = 0.5) +
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.8) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(title = "H8a: IR Memorability Across Experiment Blocks",
       x = "Experiment Phase",
       y = "IR Memorability Score") +
  theme_minimal() +
  theme(legend.position = "none")
```



```
# Graph H8b: Violin & Boxplot for WR Accuracy across Blocks
ggplot(blocked_aggregated, aes(x = Block_Number, y = WR_Accuracy, fill = Block_Number)) +
  geom_violin(trim = FALSE, alpha = 0.5) +
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.8) +
  scale_y_continuous(limits = c(0, 1.05)) + # Cap at 1.05 so the 1.0 ceiling is visible
  labs(title = "H8b: WR Accuracy Across Experiment Blocks",
       x = "Experiment Phase",
       y = "WR Accuracy") +
  theme_minimal() +
  theme(legend.position = "none")
```

```
## Warning: Removed 180 rows containing missing values or values outside the scale range
## (`geom_violin()`).
```



```
# Kruskal-Wallis Test for IR Memorability
kw_ir_block <- kruskal.test(IR_Memorability ~ Block_Number, data = blocked_aggregated)
print(kw_ir_block)
```

```
##
##  Kruskal-Wallis rank sum test
##
## data:  IR_Memorability by Block_Number
## Kruskal-Wallis chi-squared = 1.1188, df = 2, p-value = 0.5715
```

```
# Kruskal-Wallis Test for WR Accuracy
kw_wr_block <- kruskal.test(WR_Accuracy ~ Block_Number, data = blocked_aggregated)
print(kw_wr_block)
```

```
##
##  Kruskal-Wallis rank sum test
##
## data:  WR_Accuracy by Block_Number
## Kruskal-Wallis chi-squared = 0.1085, df = 2, p-value = 0.9472
```